CS Fundamentals — Complete Preparation Guide

OOP, DBMS, Operating Systems, Computer Networks — for CSE/SDE Tests & Interviews

Study Notes

Contents

1 1. Object-Oriented Programming (OOP)

1.1 1.1 Key Concepts

- **Class**: a blueprint/template defining attributes (data) and methods (behavior).
- **Object**: an instance of a class, with its own copy of instance attributes.
- **The 4 Pillars of OOP**:
  1. **Encapsulation** — bundling data and methods together, hiding internal state via access modifiers (private/protected), exposing controlled access via getters/setters.
  2. **Abstraction** — hiding implementation details, exposing only essential features (via abstract classes/interfaces).
  3. **Inheritance** — a class (child/derived) acquires properties and behavior of another class (parent/base), enabling code reuse.
  4. **Polymorphism** — the same interface behaves differently based on the object: **compile-time (method overloading)** vs **runtime (method overriding)**.

1.2 1.2 Important Notes / Interview Points

- **Overloading vs Overriding**:
  - Overloading: same method name, different parameter list, resolved at **compile time** (static binding), same class.
  - Overriding: subclass redefines a parent method with the **same signature**, resolved at **runtime** (dynamic binding), needed for runtime polymorphism.
- **Abstract class vs Interface**:
  - Abstract class can have both abstract and concrete methods, constructors, and state (fields); supports single inheritance.
  - Interface (traditionally) only declares method signatures (Java 8+ allows default/static methods); a class can implement multiple interfaces — used to achieve multiple inheritance of type.
- **Constructor**: special method called when an object is created; not inherited; can be overloaded but not overridden.
- **`this` vs `super`**: `this` refers to the current object; `super` refers to the immediate parent class (used to call parent constructor/methods).
- **Static vs Instance members**: static belongs to the class (shared across all objects); instance belongs to each object separately.
- **Composition vs Inheritance**: composition ("has-a") is generally preferred over inheritance ("is-a") for flexibility and to avoid tight coupling — a well-known design principle.
- **Diamond problem**: ambiguity when a class inherits from two classes with a common method — this is why Java disallows multiple inheritance of classes (only interfaces).

1.3 1.3 Common Interview Questions

Q: What is the difference between an abstract class and interface? A: Abstract classes allow shared implementation and state, support only single inheritance; interfaces define a contract, support multiple implementation, and traditionally hold no state (only constants).

Q: Why is Java not 100% object-oriented? A: It uses primitive types (int, char, etc.) that are not objects, for performance reasons.

Q: What is method overriding used for in real systems? A: Runtime polymorphism — e.g., a Shape base class with `area()` overridden differently by `Circle`, `Square`, `Triangle`, allowing a single interface to work over a collection of different shape objects.

Q: What is encapsulation's real benefit? A: It protects internal invariants — external code can't corrupt an object's state directly, only through validated methods, making the system

easier to maintain and debug.

1.4 1.4 Practice MCQs

Q1. Which OOP concept allows a subclass to provide a specific implementation of a method already defined in its parent class? (a) Overloading (b) Overriding (c) Encapsulation (d) Abstraction

Q2. Which of these does NOT support multiple inheritance directly in Java? (a) Interfaces (b) Classes (c) Both (d) Neither

Q3. Constructor overloading is an example of: (a) Runtime polymorphism (b) Compile-time polymorphism (c) Inheritance (d) Encapsulation

Q4. Which keyword is used to prevent a class from being inherited in Java? (a) static (b) final (c) private (d) const

Q5. In C++, which type of member function enables runtime polymorphism? (a) Static member function (b) Virtual function (c) Inline function (d) Friend function

Answer Key: 1-(b) | 2-(b) classes only support single inheritance | 3-(b) | 4-(b) | 5-(b)

2 2. Database Management Systems (DBMS)

2.1 2.1 Key Concepts

- **DBMS vs RDBMS**: RDBMS stores data in structured tables with relationships (via keys) and enforces ACID properties; plain DBMS doesn't require the relational model.
- **Keys**:
  - Primary Key — uniquely identifies each row, cannot be NULL.
  - Candidate Key — any column(s) that could qualify as primary key.
  - Foreign Key — a column referencing the primary key of another table, enforces referential integrity.
  - Composite Key — a primary key made of multiple columns.
  - Super Key — any set of columns that uniquely identifies a row (may include extra columns beyond a candidate key).
- **Normalization** — organizing data to reduce redundancy and avoid anomalies:
  - 1NF: atomic values only, no repeating groups.
  - 2NF: 1NF + no partial dependency (non-key attribute depends on the whole composite key, not part of it).
  - 3NF: 2NF + no transitive dependency (non-key attribute doesn't depend on another non-key attribute).
  - BCNF: stricter version of 3NF — every determinant must be a candidate key.
- **ACID properties**:
  - Atomicity — transaction is all-or-nothing.
  - Consistency — DB moves from one valid state to another.
  - Isolation — concurrent transactions don't interfere with each other's intermediate states.
  - Durability — once committed, changes persist even after a crash.
- **Joins**: INNER JOIN (matching rows only), LEFT JOIN (all left + matching right), RIGHT JOIN (all right + matching left), FULL OUTER JOIN (all rows from both, matched where possible).

2.2 2.2 Important Notes / Interview Points

- **Indexing**: speeds up read queries via a data structure (commonly B-Tree/B+Tree) but slows down writes (index must be updated too). Clustered index physically orders table data; non-clustered index is a separate lookup structure.
- **Denormalization**: intentionally introducing redundancy for read performance — common trade-off in read-heavy systems.
- **Transactions and Isolation Levels** (weakest to strongest): Read Uncommitted → Read Committed → Repeatable Read → Serializable. Higher isolation = fewer anomalies (dirty read, non-repeatable read, phantom read) but lower concurrency.
- **SQL vs NoSQL**: SQL databases are table-based with fixed schema, strong ACID guarantees (e.g., MySQL, PostgreSQL, Oracle); NoSQL databases (MongoDB, Cassandra, Redis) are schema-flexible, horizontally scalable, and often favor eventual consistency (BASE) over strict ACID.
- **Deadlock in DBMS**: two transactions waiting on locks held by each other — resolved via timeout or wait-for graph detection and transaction rollback.

2.3 2.3 Common Interview Questions

Q: What's the difference between DELETE, TRUNCATE, and DROP? A: DELETE removes rows (can be rolled back, logged, conditional via WHERE); TRUNCATE removes all rows instantly (minimal logging, resets identity, can't use WHERE); DROP removes the entire table structure.

Q: What is a deadlock and how is it handled? A: A cycle of transactions each waiting for a

resource locked by another. Handled via deadlock detection (wait-for graph) + rollback of one transaction, or prevention via lock ordering / timeouts.

Q: Why use normalization, and when might you denormalize? A: Normalization reduces redundancy and update anomalies. Denormalization trades some redundancy for faster reads in read-heavy systems (common in analytics/reporting).

2.4 2.4 Practice MCQs

Q1. Which normal form eliminates transitive dependency? (a) 1NF (b) 2NF (c) 3NF (d) BCNF

Q2. Which key can accept NULL values in a relational table? (a) Primary Key (b) Foreign Key (c) Composite Key (d) None of these

Q3. Which SQL JOIN returns all rows from both tables, with NULLs where there's no match? (a) INNER JOIN (b) LEFT JOIN (c) FULL OUTER JOIN (d) CROSS JOIN

Q4. Which ACID property ensures that once a transaction is committed, it stays even after a power failure? (a) Atomicity (b) Consistency (c) Isolation (d) Durability

Q5. A clustered index: (a) Creates a separate structure from data (b) Physically orders the table's data (c) Can exist multiple times per table (d) Is only used in NoSQL databases

Answer Key: 1-(c) | 2-(b) foreign keys can be NULL | 3-(c) | 4-(d) | 5-(b)

3 3. Operating Systems

3.1 3.1 Key Concepts

- **Process vs Thread**: A process is an independent program in execution with its own memory space; a thread is a lightweight unit of execution within a process, sharing the process's memory but with its own stack/registers.
- **Process states**: New → Ready → Running → Waiting (Blocked) → Terminated.
- **CPU Scheduling algorithms**:
  - FCFS (First Come First Serve) — simple, can cause convoy effect.
  - SJF (Shortest Job First) — minimizes average waiting time, but can starve long jobs.
  - Round Robin — time-quantum based, fair, good for time-sharing systems.
  - Priority Scheduling — can cause starvation, solved via aging.
- **Memory Management**:
  - Paging — divides memory into fixed-size pages/frames, eliminates external fragmentation but can cause internal fragmentation.
  - Segmentation — divides memory into variable-size logical segments (code, stack, heap), can cause external fragmentation.
  - Virtual Memory — allows executing processes larger than physical memory using disk (swap space) + paging.
- **Deadlock**: occurs when 4 conditions hold simultaneously — Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait. Prevent by breaking any one condition.
- **Synchronization primitives**: Mutex (mutual exclusion lock, ownership-based), Semaphore (counter-based signaling, can allow N threads), Monitor (higher-level construct combining mutex + condition variables).

3.2 3.2 Important Notes / Interview Points

- **Race condition**: occurs when multiple threads access shared data concurrently and the outcome depends on timing/order — fixed via synchronization (locks).
- **Context switching**: saving the state of a running process/thread and loading another; has overhead, minimized by efficient scheduling.
- **Thrashing**: system spends more time swapping pages than executing — caused by overcommitting processes relative to available RAM.
- **Belady's Anomaly**: in FIFO page replacement, increasing the number of frames can sometimes *increase* page faults (counter-intuitive).
- **Starvation vs Deadlock**: starvation = a process waits indefinitely (but system still progresses); deadlock = system is stuck completely (no process progresses).

3.3 3.3 Common Interview Questions

Q: What's the difference between a process and a thread, in practice? A: Processes are isolated (separate memory) and heavier to create/switch; threads share memory within a process, making communication faster but requiring careful synchronization to avoid race conditions.

Q: How do you prevent deadlock? A: Break one of the 4 necessary conditions — e.g., impose a global lock-acquisition order (breaks circular wait), or use timeouts (breaks hold-and-wait).

Q: What is virtual memory and why is it useful? A: An abstraction letting processes use more memory than physically available RAM by swapping pages to/from disk, and giving each process an isolated address space.

3.4 3.4 Practice MCQs

Q1. Which scheduling algorithm can lead to starvation of long processes? (a) Round Robin (b) FCFS (c) SJF (d) None

Q2. A binary semaphore is functionally similar to a: (a) Monitor (b) Mutex (c) Spinlock only (d) Deadlock detector

Q3. Which of the following is NOT a necessary condition for deadlock? (a) Mutual Exclusion (b) Hold and Wait (c) Preemption (d) Circular Wait

Q4. Thrashing occurs due to: (a) Too few processes (b) Excessive page faults due to over-commitment of memory (c) Efficient CPU scheduling (d) Small page size

Q5. Which memory management technique can cause external fragmentation? (a) Paging (b) Segmentation (c) Both (d) Neither

Answer Key: 1-(c) | 2-(b) | 3-(c) it's "No Preemption" that's the condition, not "Preemption" | 4-(b) | 5-(b)

4 4. Computer Networks

4.1 4.1 Key Concepts

- **OSI Model (7 layers, top to bottom)**: Application, Presentation, Session, Transport, Network, Data Link, Physical. Mnemonic: "All People Seem To Need Data Processing."
- **TCP/IP Model (4-5 layers)**: Application, Transport, Internet, Network Access (Link) — a simplified, practically-used model.
- **TCP vs UDP**:
  - TCP: connection-oriented, reliable (acknowledgments + retransmission), ordered delivery, flow/congestion control — used for HTTP, file transfer.
  - UDP: connectionless, no reliability guarantee, faster/lower overhead — used for streaming, DNS, gaming.
- **Three-way handshake (TCP connection setup)**: SYN → SYN-ACK → ACK.
- **IP Addressing**: IPv4 (32-bit, e.g. 192.168.1.1), IPv6 (128-bit). Subnetting divides a network into smaller sub-networks using a subnet mask.
- **DNS**: translates domain names to IP addresses.
- **HTTP vs HTTPS**: HTTPS adds TLS/SSL encryption on top of HTTP for secure communication.
- **Common ports**: HTTP=80, HTTPS=443, FTP=21, SSH=22, DNS=53, SMTP=25.

4.2 4.2 Important Notes / Interview Points

- **Switch vs Router vs Hub**: Hub broadcasts to all ports (no intelligence); Switch forwards frames based on MAC address (Layer 2); Router forwards packets between networks based on IP address (Layer 3).
- **Congestion control vs Flow control**: Flow control prevents a fast sender from overwhelming a slow receiver; congestion control prevents overwhelming the network itself (e.g., TCP's slow start, congestion avoidance).
- **Latency vs Bandwidth**: Latency = time for a single packet to travel; Bandwidth = maximum data transfer rate/capacity of the link. Low latency ≠ high bandwidth, and vice versa.
- **Load balancing** distributes incoming requests across multiple servers to improve availability/scalability — common in interview system-design discussions.

4.3 4.3 Common Interview Questions

Q: Why does TCP need a 3-way handshake? A: To synchronize sequence numbers between client and server and confirm both sides are ready to communicate reliably before data transfer begins.

Q: When would you choose UDP over TCP? A: When speed matters more than reliability and occasional data loss is acceptable — e.g., video streaming, live gaming, DNS lookups.

Q: What happens when you type a URL into a browser? *(classic systems question)* A: DNS resolution (domain→IP) → TCP connection (3-way handshake) → TLS handshake (if HTTPS) → HTTP request sent → server processes and responds → browser renders the response.

4.4 4.4 Practice MCQs

Q1. Which layer of the OSI model is responsible for routing? (a) Data Link (b) Network (c) Transport (d) Session

Q2. Which protocol is connection-oriented and reliable? (a) UDP (b) IP (c) TCP (d) ICMP

Q3. The default port for HTTPS is: (a) 80 (b) 21 (c) 443 (d) 22

Q4. A device that forwards data based on MAC address is a: (a) Router (b) Switch (c) Hub (d) Repeater

Q5. DNS is primarily responsible for: (a) Encrypting data (b) Translating domain names to IP addresses (c) Routing packets (d) Error correction

Answer Key: 1-(b) | 2-(c) | 3-(c) | 4-(b) | 5-(b)

5 5. Data Structures — Quick Overview

(Full detailed DSA guide with problems, code, and approach is a separate document — this section is a fast conceptual refresher for test/interview MCQs.)

5.1 5.1 Key Concepts & Complexities

Structure	Access	Search	Insert	Delete	Notes
Array	O(1)	O(n)	O(n)	O(n)	Fixed size, contiguous memory
Linked List	O(n)	O(n)	O(1)*	O(1)*	*at known position; dynamic size
Stack	O(n)	O(n)	O(1)	O(1)	LIFO — push/pop at top
Queue	O(n)	O(n)	O(1)	O(1)	FIFO — en-queue/dequeue
Hash Table	—	O(1) avg	O(1) avg	O(1) avg	O(n) worst case with collisions
Binary Search Tree	O(log n) avg	O(log n) avg	O(log n) avg	O(log n) avg	O(n) worst if unbalanced
Balanced BST (AVL/Red-Black)	O(log n)	O(log n)	O(log n)	O(log n)	Guaranteed balance
Heap (Binary)	—	O(n)	O(log n)	O(log n)	Min/max at root in O(1)
Graph (Adjacency List)	—	O(V+E)	O(1)	O(E)	Space-efficient for sparse graphs

5.2 5.2 Important Notes

- **Array vs Linked List**: arrays offer O(1) random access but costly insert/delete (shifting); linked lists offer O(1) insert/delete at known nodes but O(n) access.
- **Stack uses**: function call stack, undo operations, expression evaluation (infix→postfix), backtracking (DFS uses a stack implicitly via recursion).
- **Queue uses**: BFS traversal, task scheduling, buffering (e.g., IO buffers), print queues.
- **Hashing collisions** resolved via chaining (linked list per bucket) or open addressing (linear/quadratic probing, double hashing).

5.3 5.3 Practice MCQs

Q1. Which data structure is used to implement recursion internally? (a) Queue (b) Stack (c) Array (d) Heap

Q2. What is the worst-case time complexity of searching in an unbalanced BST? (a) O(1) (b) O(log n) (c) O(n) (d) O(n log n)

Q3. Which traversal of a graph uses a queue? (a) DFS (b) BFS (c) Both (d) Neither

Q4. In a min-heap, the smallest element is located at: (a) The last leaf (b) The root (c) Any leaf (d) The middle

Answer Key: 1-(b) | 2-(c) | 3-(b) | 4-(b)

6 Quick Revision — Interview One-Liners

- **Encapsulation**: hide data, expose behavior via controlled interface.
- **Polymorphism**: overloading = compile-time, overriding = runtime.
- **Normalization**: 1NF (atomic) → 2NF (no partial dependency) → 3NF (no transitive dependency).
- **ACID**: Atomicity, Consistency, Isolation, Durability.
- **Deadlock conditions**: Mutual Exclusion + Hold & Wait + No Preemption + Circular Wait.
- **TCP vs UDP**: reliable/ordered vs fast/connectionless.
- **Process vs Thread**: isolated memory vs shared memory within a process.
- **Paging vs Segmentation**: fixed-size blocks vs variable-size logical units.

End of CS Fundamentals Guide. Revisit this the night before your test/interview — it's designed as a fast-recall sheet once you've studied each topic in depth once.